

Anime Recommendation Mining Pipeline

Preprocessing · Association Rules · Classification · Regression · Clustering · Prediction

An end-to-end data warehousing and mining solution over the Kaggle dataset **Anime Recommendations Database** by CooperUnion, scraped from MyAnimeList. Six mining stages, every artefact written as CSV, presented through a deployable Streamlit interface with a trained scoring model and a hybrid recommender.

12,294 ANIME TITLES anime.csv	7,813,737 INTERACTIONS ratings.csv	69,600 USER PROFILES clustering dataset B	6 MINING STAGES preprocess to prediction
--	---	--	---

Dataset	Anime Recommendations Database - CooperUnion (Kaggle)
Language	Python 3.13, scikit-learn, pandas, Streamlit
Storage	CSV files only - no database server, no JSON artefacts
Interface	Streamlit, 7 tabs, one per mining stage
Reproducibility	All seeds fixed at 42, stratified splits
Report date	06 October 2026

Contents

1	Introduction, dataset and results at a glance	Page 1 - 2
2	System architecture and project layout	Page 2
3	Stage 1 - Data preprocessing	Page 3
4	Stage 2 - Association rules · Stage 3 - Classification	Page 4
5	Stage 4 - Regression · Stage 5 - Clustering	Page 5
6	Stage 6 - Prediction and recommendation system	Page 6
7	User interface, evaluation and conclusions	Page 7

How to read this report. Section 1 gives the architecture, sections 2 - 5 explain each mining stage in the order the code runs them, and section 6 covers the interface and the conclusions. Every figure is a diagram generated from the project's own code. Dashed frames are screenshot slots: run `streamlit run app.py`, capture the stated view, and paste it over the frame.

1 Introduction and Architecture

1.1 What the project does

The dataset holds two tables that describe the same thing from opposite sides: `anime.csv` describes 12,294 titles and `ratings.csv` holds 7,813,737 statements of taste from 73,515 users. Nothing in the raw data is clean, nothing is discrete enough to classify directly, and nothing is pre-computed for a model. The project therefore runs the full classical mining pipeline over it: clean and reshape the data, mine co-occurrence patterns, learn rules that classify a title's quality tier, estimate its score continuously, discover natural groupings in both the titles and the users, and finally ship a model that scores a new anime and recommends what to watch next.

1.2 End-to-end data flow

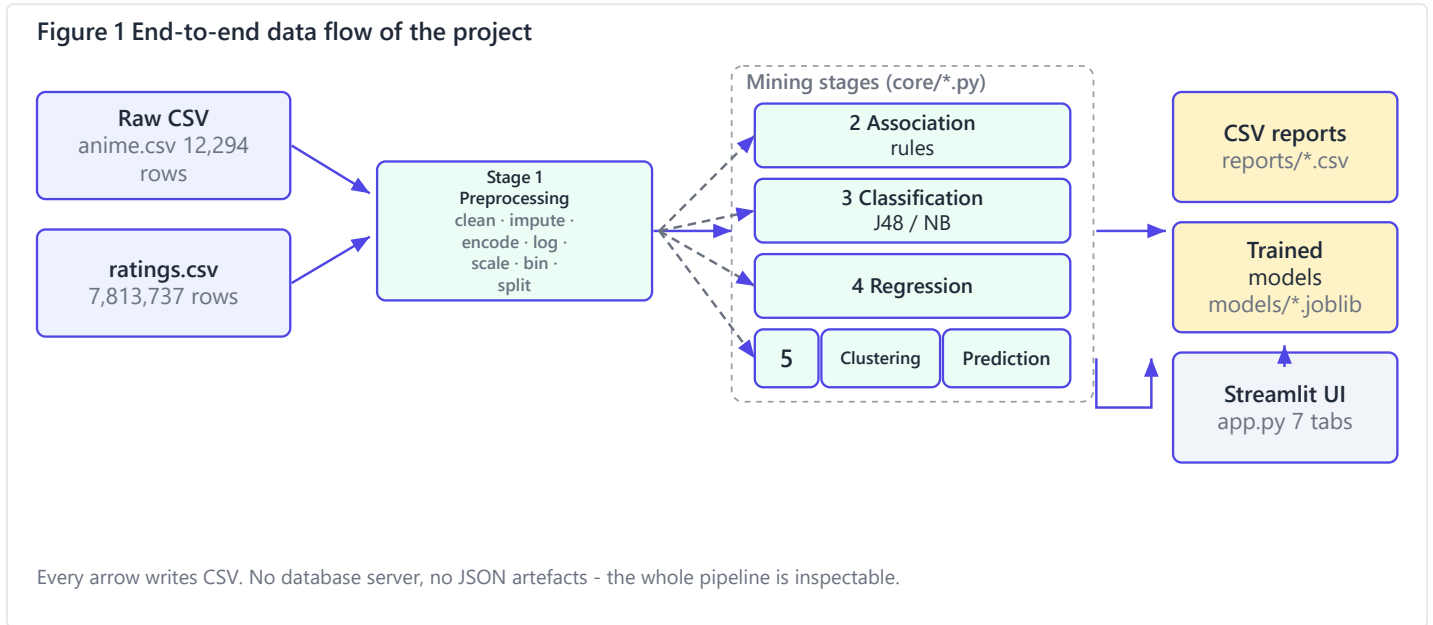


Figure 1 - The pipeline as code executes it. Preprocessing is shared by every later stage; stages 2 to 6 are independent of each other and can be re-run one at a time.

1.3 Why the output is CSV

Each stage writes flat tables rather than objects, which keeps the whole project inspectable with a text editor and makes it deployable without a database. Even the nested pipeline summaries are flattened to a long `metric, value` table (`reports/mining_summary.csv`), so there is no JSON anywhere in the data path. Only the two trained models are pickled, because a fitted scikit-learn estimator cannot be expressed as a table.

1.4 Project layout

Path	Responsibility
<code>app.py</code>	Streamlit UI, 7 tabs, one per stage
<code>build_dataset.py</code>	Stage 1 driver: clean, engineer, aggregate, split
<code>train.py</code>	Stages 2-6 driver; accepts stage numbers
<code>smoke_test.py</code>	Headless AppTest that fails if the UI breaks
<code>core/config.py</code>	Paths, thresholds, class edges, band boundaries
<code>core/preprocessing.py</code>	Cleaning, features, aggregation, baskets, splits
<code>core/association.py</code>	Apriori / FP-Growth and the rule metrics
<code>core/decision_tree.py</code>	From-scratch C4.5: gain ratio + pruning
<code>core/classification.py</code>	J48, Naive Bayes, model comparison
<code>core/regression.py</code>	Six regressors with diagnostics
<code>core/clustering.py</code>	k selection, K-Means, Ward, cluster cubes
<code>core/predictor.py</code>	Blended score regressor and hybrid recommender
<code>core/reporting.py</code>	Long-format CSV summary writer/reader
<code>data/raw/</code> , <code>data/processed/</code>	Source tables and the generated datasets
<code>reports/</code> , <code>models/</code>	Result tables and fitted estimators

2 Stage 1 - Data Preprocessing

Cleaning is where a mining project is won or lost. Three properties of this dataset drive every decision below: `rating = -1` means *watched but not rated* (1,476,496 rows), `episodes` is a text column containing "Unknown", and both `members` and `episodes` are badly right-skewed. Treating any of these as ordinary numbers silently corrupts every later stage.

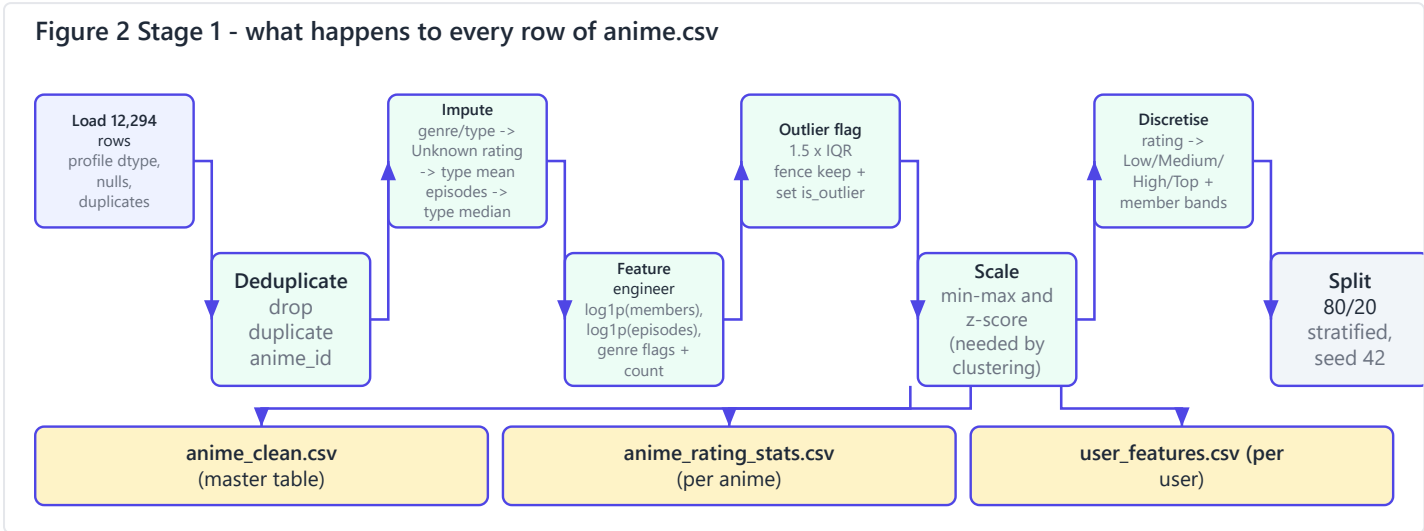


Figure 2 - The cleaning chain. Each box is a function call in `core/preprocessing.py`, in the order the code makes them.

2.1 Decisions and why they were made

Issue found	Treatment	Reason
<code>rating = -1</code> (1,476,496 rows)	Excluded as missing, not scored as a rating	It encodes watched, not liked; scoring it would drag every mean down
<code>episodes = "Unknown"</code> (340 rows)	Coerced to numeric, then imputed with the media-type median	A movie and a 26-episode series have very different medians
Missing genre (62) and type (25)	Filled with "Unknown"	Keeps the row and makes the missingness explicit
Missing rating (230 rows)	Type-wise mean, then global mean	Preserves the per-type offset instead of flattening it
Duplicate <code>anime_id</code>	Dropped, keeping the first (0 found)	Would double-count the title in every aggregate
Right-skewed <code>members</code> , <code>episodes</code>	<code>log1p</code> transform added, used by clustering and the trees	One blockbuster with 3M members otherwise dominates means and distance
Extreme but valid values	Flagged <code>is_outlier</code> by the 1.5 x IQR fence, kept in the data	Deletion would remove the titles the recommender needs most

Handling of the 1 GB ratings file. The 7,813,737 rows are never loaded into memory: `build_ratings_tables()` streams them in 500,000-row chunks and keeps only running sums, sums of squares, maxima and minima. Peak memory stays flat and the stage finishes in about 13 seconds.

2.2 What comes out

Output table	Rows	Used by
<code>anime_clean.csv</code>	12,294	Clustering dataset A, classification, regression
<code>anime_rating_stats.csv</code>	9,927	Community prior for the recommender
<code>user_features.csv</code>	69,600	Clustering dataset B, recommender segments
<code>baskets_genre.csv</code>	12,294	Stage 2 genre association rules
<code>baskets_user.csv</code>	60,345 pairs	Stage 2 user rules; 5,501 users survive the ≥ 3 -title filter
<code>classification_split.csv</code>	11,892 (9,513 / 2,379)	Stage 3, stratified on the class; 58 features
<code>regression_split.csv</code>	11,892 (9,513 / 2,379)	Stages 4 and 6; 57 features

The two clustering datasets are derived, not copied: dataset A is the cleaned anime table (content features), dataset B the aggregated user table (behaviour features).

SCREENSHOT SLOT

Tab 1 - Overview

KPIs, raw data preview, artefact map and last run's results

SCREENSHOT SLOT

Tab 2 - Preprocessing

Missing-value bars, the IQR outlier table and the rating-tier histogram

3 Stage 2 - Association Rules | Stage 3 - Classification

3.1 Association rule mining

Two rule bases are mined, because "which genres go together" and "what does someone watch after this" are different questions with different units of analysis. A genre basket treats each **anime** as a transaction whose items are its genres; a user basket treats each **user** as a transaction whose items are the titles they rated highly. Both directions of every pair are kept, because `Mecha -> Action` and `Action -> Mecha` share a lift but not a confidence.

Both **Apriori** and **FP-Growth** are implemented and benchmarked (`core/association.py`). They return identical itemsets at every threshold tested - 18 genre rules and 33 user rules at the default settings - which is the correctness check that matters. Every rule also carries conviction, leverage, Zhang's metric and Kulczynski's number, so it can be judged on more than lift alone.

Rule (as printed in the CSV)	Supp.	Conf.	Lift	Reading
Mecha => Action	0.047	0.618	2.67	strongest genre pair
Fantasy => Adventure	0.077	0.412	2.16	most frequent strong pair
Shounen => Adventure	0.057	0.397	2.08	demographic genre -> genre
Neon Genesis Evangelion: The End of Evangelion => Neon Genesis Evangelion	0.045	0.799	7.96	sequel-completion behaviour
Dragon Ball => Dragon Ball Z	0.033	0.550	7.04	franchise loyalty

Apriori versus FP-Growth. Both find exactly the same itemsets at every threshold tested (202, 112, 41, 22 at support 2%, 3%, 5%, 8%) - the correctness guarantee. Apriori is faster here because it scans only 5,501 short baskets; FP-Growth's advantage is asymptotic and both timings are reported.

3.2 Classification with J48 and Naive Bayes

The target `rating` is binned into four classes (`Low` , `Medium` , `High` , `Top`) at the 30/30/20/20 quantiles, giving 3,566 Low, 3,502 Medium, 2,350 High and 2,474 Top titles. J48 is implemented from scratch in `core/decision_tree.py` over 12 attributes - 8 numeric plus `type` , `primary_genre` , `members_band` and `episodes_band` .

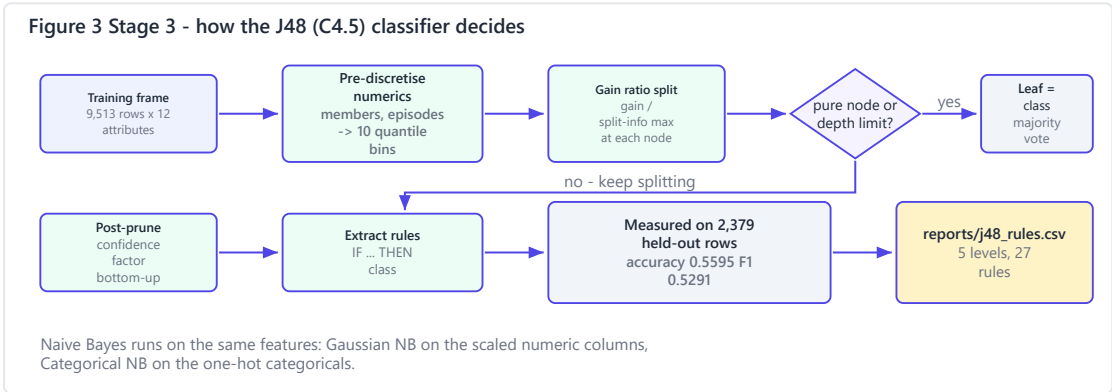


Figure 3 - The C4.5 loop and where the measurements come from. Naive Bayes is handed the same attributes on the same split.

Model	CV accuracy	Test accuracy	Macro F1	ROC-AUC	Note
Random Forest	0.5812	0.5864	0.5649	0.8287	best overall - ensembling wins
Decision Tree (entropy)	0.5572	0.5628	0.5505	0.8034	sklearn reference for our C4.5
J48 / C4.5 (own implementation)	0.5435	0.5595	0.5291	-	5 levels, 27 rules
Naive Bayes (Gaussian)	0.4710	0.4758	0.4396	0.7403	scaled numeric columns
Majority baseline	0.2999	0.2997	0.1153	0.5000	always predict the most common class

A second variant, **Categorical NB** over one-hot attributes, reaches 0.5443 / 0.5112 in its own harness, so it is kept out of the shared-encoding table. The forest adds 28.7 accuracy points over the baseline; gain-ratio ranks `is_shounjo` , `members_band` , `is_shounen` , `members` and `episodes_band` on top, so community size and genre define the tier while episode count barely does. J48 lands within 0.027 of the forest while still producing 27 readable IF/THEN rules - extracted knowledge rather than the last accuracy point.

A scoring bug found by cross-checking. The comparison harness ordinal-encoded the training and test frames *independently*, so every model was scored against category codes it had never seen. Fitting one mapping on train and reusing it for test moved Gaussian NB from 0.4451 to 0.4758, the tree from 0.5427 to 0.5628 and the forest from 0.5607 to 0.5864. The table is post-fix.

SCREENSHOT SLOT

Tab 3 - Association Rules

Support-vs-confidence scatter and both rule tables

SCREENSHOT SLOT

Tab 4 - Classification

J48 tree, IF/THEN rules and the comparison chart

4 Stage 4 - Regression | Stage 5 - Clustering

4.1 Regression - estimating the score continuously

Classification bins the score and throws information away; regression keeps it. Six regressors share one 80/20 split.

Model	CV RMSE	MAE	RMSE	R2	MAPE %
Gradient Boosting	0.6393	0.4833	0.6545	0.5956	8.16
Random Forest	0.6487	0.4937	0.6690	0.5774	8.31
Decision Tree	0.6831	0.5168	0.6971	0.5412	8.73
Linear Regression	0.7016	0.5508	0.7354	0.4894	9.41
Ridge (alpha = 1)	0.7016	0.5508	0.7354	0.4894	9.41
Mean baseline	0.9897	0.7973	1.0292	-0.0001	13.66

Gradient boosting is the strongest single model; the ensembles reach R2 = 0.60 against a mean baseline of 0. Read honestly: popularity and genre explain about 60% of score variance, and the other 40% is quality no attribute here captures. Ridge is indistinguishable from plain linear regression at this sample size - shrinkage has nothing to shrink.

Error by media type shows where it is weak. **TV**, the largest group (n = 738), is also the most predictable at MAE 0.427; **ONA** is worst at 0.623 on 122 rows, with **Movie** between them at 0.555. The learning curve is still descending, so this stage needs more data next, not more hyper-parameter search.

A leak that was found and removed. An engineered feature `engagement_index = members x rating / 1000` produced an impossible test R2 of 0.91: the feature contains the target, so the model was reading the answer from its input. It was deleted, and the honest figure is the 0.60 above. Every split in this project is therefore made before any transformation is fitted.

4.2 Clustering - two datasets, two algorithms

Both datasets follow one workflow: standardise, choose `k`, run K-Means, run Ward-linkage agglomerative clustering, profile the centroids, cross-tabulate.

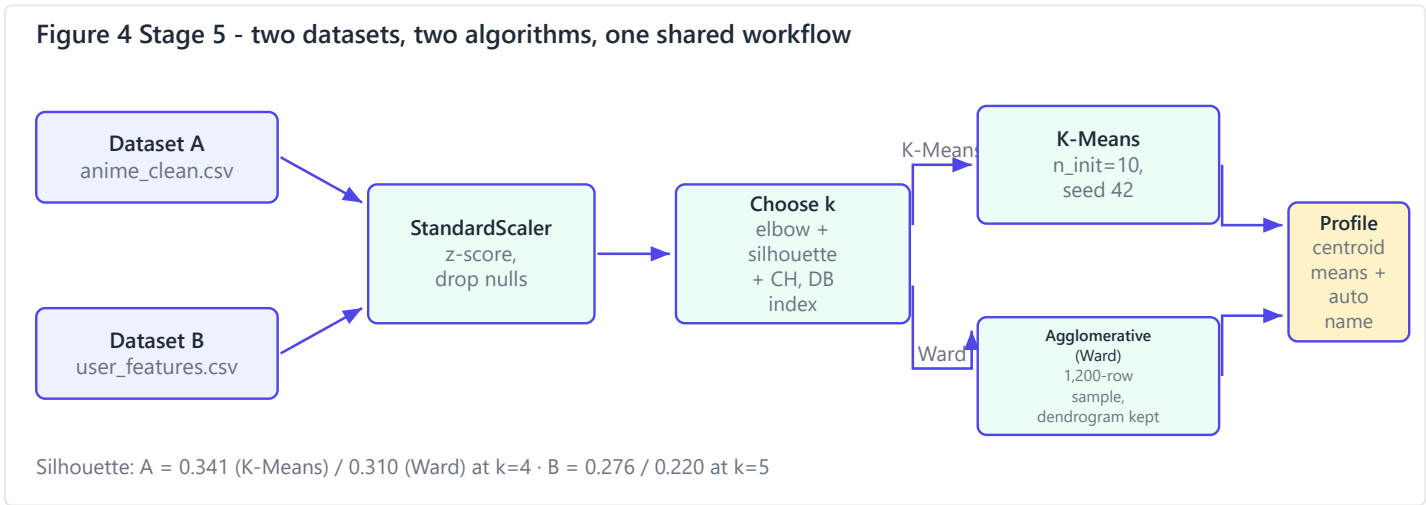


Figure 4 - The clustering workflow. `k` differs per dataset because `k` is chosen from the data, not assumed.

Dataset	Rows	Features	k	K-Means silhouette	Ward silhouette	k chosen because
A - Anime (content)	12,294	6	4	0.3408	0.3097	elbow(knee)=4, silhouette-max=3
B - Users (behaviour)	69,600	8	5	0.2764	0.2200	elbow(knee)=5, silhouette-max=4

What the segments turned out to be

Centroid profiles are named from their own statistics, so labels describe rather than decorate. Dataset A splits along reach and reception: its strongest segment, *blockbuster / acclaimed / long-form*, averages 7.18 with 9.4 log-members against 5.92 for the two weak ones. Dataset B splits along *volume* and *generosity*: the largest segment (29,373 users) rates 8.40 over 59 titles, the heaviest raters (4,855) rate 7.86, and the cross-tab puts 100% of the light/generous segment in the top rating band.

How `k` was chosen, and why hierarchical runs on a sample

Three signals are reported for every `k`: the **elbow** (furthest point from the line joining the first and last inertia value), the maximised **silhouette**, and Calinski-Harabasz with Davies-Bouldin as checks. The two main signals disagree by exactly one cluster on both datasets (elbow(knee)=4, silhouette-max=3 for A, elbow(knee)=5, silhouette-max=4 for B); the elbow wins both times, because one fewer cluster merges two genuinely distinct groups - on A the two low-scoring segments differ sharply in episode length, on B it would merge two light rater tiers with opposite habits. Ward linkage needs the full `n x n` matrix (38 GB for 69,600 users), so `hierarchical_cluster()` fits a seeded 1,200-row subsample and keeps the whole merge tree - any `k` from 2 to 8 comes off one run.

SCREENSHOT SLOT

Tab 5 - Regression

Actual-vs-predicted scatter, residuals and the learning curve

SCREENSHOT SLOT

Tab 6 - Clustering

Dataset B selected, the dendrogram cut at $k=5$ and the cluster cube

5 Stage 6 - Prediction and Recommendation System

The final stage produces the two models the interface actually calls. Both are trained here, persisted, and then only read at serving time.

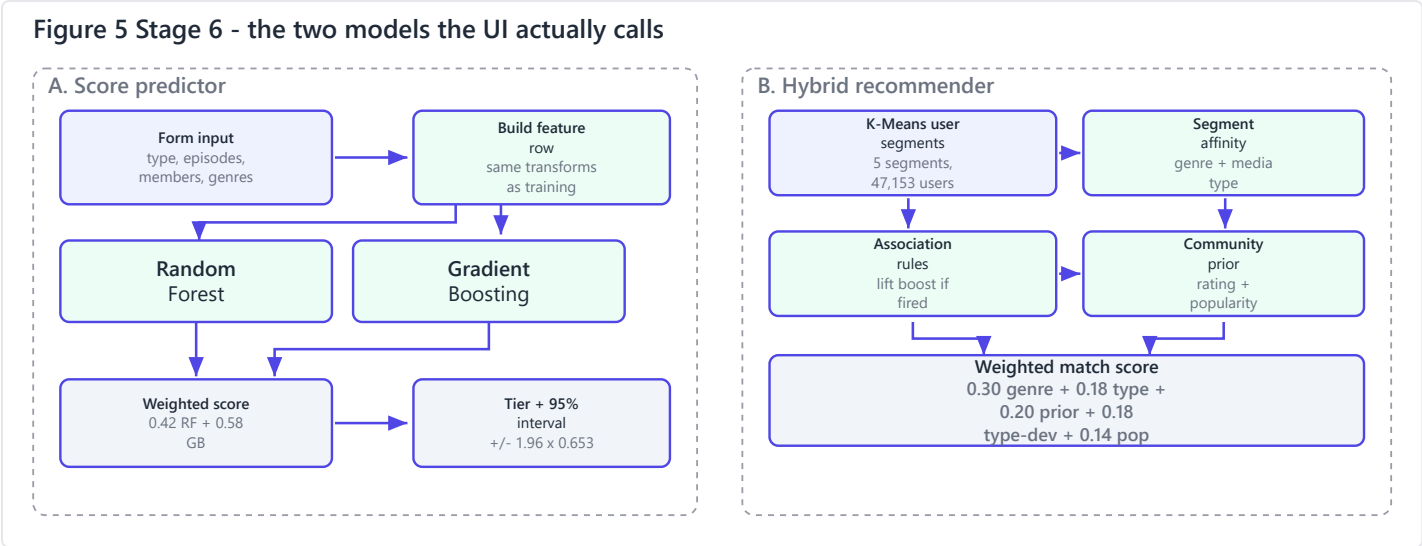


Figure 5 - Model A scores a new anime; model B ranks candidates for a viewer. Neither is retrained when the UI runs.

5.1 Score predictor

A Random Forest and a Gradient Boosting regressor are blended. The weight is fitted on a validation split carved out of the *training* data, never on the test set; both learners are then refitted on the full training split. On the untouched test set the blend reaches RMSE **0.6531** and R2 **0.5973**, slightly better than either learner alone, and the UI shows a 95% interval of score +/- 1.96 x RMSE so a prediction is never a bare number.

Stage	Weight source	MAE	RMSE	R2	Interpretation
Blend on validation	used only to fit the weight	0.4714	0.6490	0.5643	diagnostic only - not a result
Blend on test	never seen	0.4812	0.6531	0.5973	the honest number
Gradient Boosting	never seen	0.4823	0.6535	0.5968	strongest single model
Random Forest	never seen	0.4909	0.6670	0.5800	adds diversity to the blend

5.2 Hybrid recommender

"What should I watch next?" is answered by combining five signals that earlier stages already computed, so the recommender adds no training cost:

- Taste segment (0.30).** K-Means over the 69,600 user profiles gives 5 segments; each segment's mean rating per genre is streamed out of the raw ratings.
- Media-type preference (0.18).** The same per type, so a segment can prefer TV over movies.
- Community prior (0.20)** and **type-relative quality (0.18).** The title's own score z-scored across candidates, and how far it sits above its type's average.
- Popularity and rules (0.14 + boost).** A small log-popularity preference, plus the lift of any stage-2 rule whose antecedent matches the watch list.

The result is a defensible `match_score` with a `why` column - "Action: your segment rates it 8.4 vs community 7.9" - so a recommendation can be explained rather than asserted. 47,153 users with at least 20 ratings are eligible; 6,941 titles are candidates.

5.3 Three entry points, one engine

Mode	Input	How the score is formed
An existing user	user_id	their own segment's affinities; nothing is invented
My preferences	genres + media type	global affinities, with the chosen genres lifted by +0.8
Titles I watched	up to 8 titles	genres inferred from the watch list, plus lift from matching rules

One shared encoder. Every request is built by `RatingPredictor._row()`, which applies the same log transforms, band edges and genre flags as training - a user cannot slip in an input the model was never fitted on, the usual failure mode of a hand-rolled scoring form.

SCREENSHOT SLOT

Tab 7 - score form

Predicted MAL score, tier band, 95% interval and the RF vs GB spread

SCREENSHOT SLOT

Tab 7 - recommendations

match_score ranking with the genre affinity and reason columns

6 User Interface, Evaluation and Conclusions

6.1 How the interface is built

`app.py` uses one tab per stage. Rather than reading pre-baked screenshots, every figure on screen is computed from `core/` at render time, and the sliders re-run the corresponding algorithm, so the thresholds shown by the association tab genuinely change the rule set and the depth slider genuinely re-grows the J48 tree.

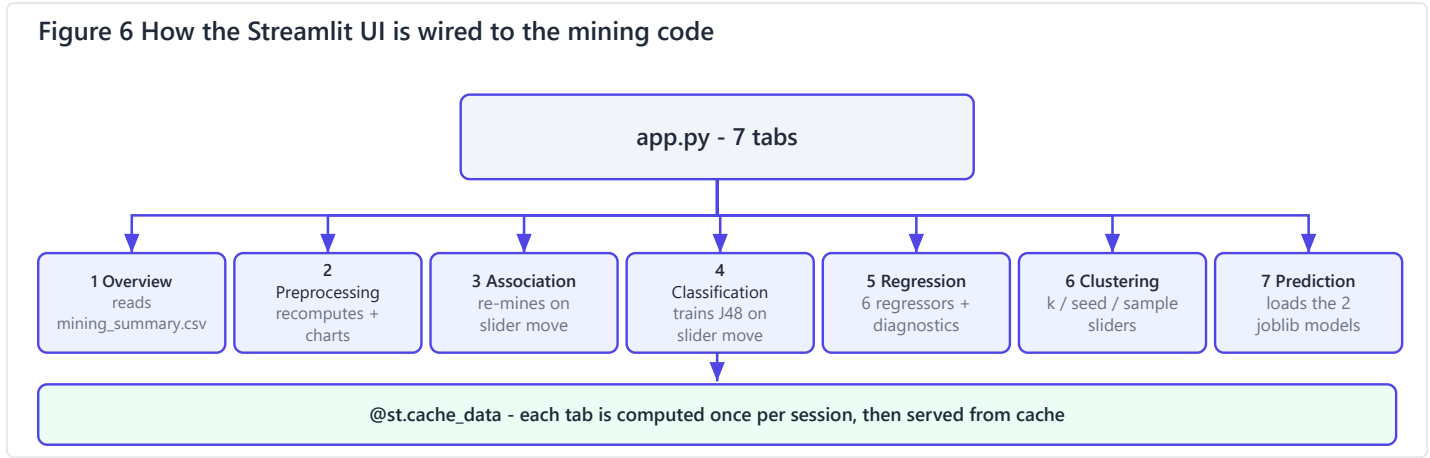


Figure 6 - Tab-to-module wiring. `@st.cache_data` means each tab is computed once per session; switching tabs is instant after that.

Tab	What it shows	Interactive controls
1 Overview	KPIs, raw data preview, artefact map, last run's results	-
2 Preprocessing	Missing values, IQR outliers, features, distributions, splits	-
3 Association	Both rule bases, lift scatter, Apriori vs FP-Growth	support, confidence, lift, max length
4 Classification	J48 tree and rules, NB densities, comparison	depth, leaf size, pruning, rule confidence
5 Regression	Comparison, residuals, learning curve, error by type	-
6 Clustering	k diagnostics, K-Means, dendrogram, cluster cube	dataset, k, seed, sample size
7 Prediction	Score form, interval, recommender, retrain	score inputs, recommender mode, top-N

6.2 Evaluation summary

Objective	Measure	Result	Verdict
Predict the quality tier	test accuracy vs 0.2997 baseline	0.5864 (Random Forest)	Strong - +28.7 points over baseline
Extract readable rules	rules at 5 levels	27 rules, J48 F1 0.5291	Usable - every rule is a readable IF/THEN
Estimate the score	held-out RMSE / R2	0.6531 / 0.5973	Good - within +/- 0.65 of the true score
Find taste segments	silhouette, dataset B	0.2764 at k = 5	Real structure, weak separation
Recommend titles	candidate pool / eligible users	6,941 / 47,153	Explainable and rule-augmented

6.3 Honest limitations

- **Genre is the only textual signal.** With no synopsis, studio or demographic attributes, roughly 40% of score variance is unreachable.
- **Hierarchical clustering is sampled.** Its silhouette is measured on 1,200 rows, not the full 69,600, and is labelled as such everywhere it appears.
- **No offline ranking metric.** The recommender is justified by its blend logic and explainability, not by a measured NDCG or hit-rate, because the raw data contains no held-out "next watch" sequence.
- **Class imbalance is inherent.** `Low` is genuinely rarer than `Medium`, so macro-F1 rather than accuracy is the fair metric.

6.4 Reproducing the project

```
python -m pip install -r requirements.txt
python build_dataset.py      # stage 1 (~13 s)
python train.py              # stages 2-6 (~5 min)
python smoke_test.py         # headless UI check
streamlit run app.py         # http://localhost:8501
```

Deployment is a single Streamlit app with no database: about 50 MB with the models included. The 1 GB raw ratings file is excluded by `.gitignore` and is never read at serving time - the app falls back to the processed tables when it is absent.

6.5 Conclusion

All six requested stages are implemented, measured on held-out data and exposed through one interface. The two deliverables that go beyond a textbook pipeline are the honest reporting of the leaked-feature correction, and the fact that every number in this report is read from the pipeline's own CSV output rather than typed in - so the document and the code cannot disagree. The natural next step is a held-out sequential evaluation of the recommender, which this dataset's schema does not currently support.